AutoBayes milestone M0: evidence for review

Claude session, branch `m0/gaussian-varying-intercept`

September 8, 2026

1 What M0 delivers

One executable path for the Gaussian varying-intercept model

$$y_i \sim \text{Normal}(\mu_i, \sigma), \qquad \mu_i = a_c + (x_i - \bar{x})\, b + u[g_i], \qquad u[j] = s_g\, z_j, \quad z_j \sim \text{Normal}(0, 1),$$

with explicit priors on the four brms classes (`Intercept`, `b`, `sd`, `sigma`), an inspectable PyMC model, posterior and predictive draws, PyMC's NUTS on the CPU, and a JAX/NumPyro path with an explicit device choice. Everything outside that family is refused with an error naming the construct.

Branch, clone	`m0/gaussian-varying-intercept` in `IQSS-Research/projects/AutoBayes`, five commits on `f87f639`
Package	`autobayes/` (`formula.py`, `priors.py`, `model.py`, `inference.py`, `predict.py`, `diagnostics.py`)
Tests	`tests/`: 36 fast, 2 slow (`-m slow`); all 38 pass
Reference density	`tests/reference_logp.py`, imports nothing from `autobayes`, PyMC, or Bambi
Example	`examples/m0_varying_intercept.py`; outputs captured in `docs/m0-evidence/`
Decision record	`docs/m0-decision.md` (written before implementation), `docs/coordination/STATUS.md`
Environment	`.venv`: pymc 5.28.5, bambi 0.17.2, numpyro 0.21.0, jax 0.10.2 (CPU wheel), arviz 0.23.4, pytensor 2.38.3

2 Bambi already builds this model; the layer is thin on purpose

The milestone asked for a comparison before any new code. Verified against the installed Bambi 0.17.2: it parses `y ~ x + (1|g)`, builds the one-hot group matrix with levels in sorted order, accepts explicit priors including a hyperprior on the group SD, uses the non-centred parameterisation by default (as brms always does), centres the predictors by default and back-transforms the intercept after sampling (as brms does), exposes the PyMC model as `model.backend.model` after `build()`, forwards `**kwargs` to `pm.sample` so `nuts_sampler="numpyro"` with `nuts_sampler_kwargs={"chain_method": ...}` reaches PyMC 5.28.5's `sample_jax_nuts`, and predicts for new group levels with brms's default `sample_new_levels="uncertainty"` semantics only.

The layer therefore adds six things, each because Bambi does not do it: (1) a grammar check that refuses `(1 + x | g)`, `(1 | g/h)`, two grouping factors, interactions, transformations, `0 +`, categorical predictors, and missing data with a message naming the construct; (2) mandatory explicit priors by class, with no silent defaults; (3) a `describe()` that states the density actually sampled, centring and transforms included; (4) device resolution that raises rather than falling back, enables float64, sets the host device count for parallel CPU chains, and reports the actual device; (5) predictive draws with named new-group semantics, including the population option (brms `"gaussian"`)

that Bambi lacks; (6) the diagnostic panel with the thresholds from `reference/diagnostics.md`. No parser, design-matrix builder, prior language, or sampler was written.

3 The model that is actually sampled

`model.describe()` on the example data prints the specification below; the PyMC representation follows it. Both are real output (`example_pymc_cpu.txt`, `pymc_model_repr.txt`).

```
Linear predictor
  mu_i = a_c + (x_x - mean(x_x)) * b_x + u[school_i]
  Predictors are centred at their sample means (center_predictors=True, as in
  brms and Bambi). a_c is the expected response at the predictor means; the
  prior on the intercept applies to a_c. The posterior reports the raw-scale
  intercept a = a_c - sum_p mean(x_p) * b_p, computed after sampling.
Group effects (non-centred)
  u[j] = sd_school * z[j],  z[j] ~ Normal(0, 1),  j = 0..19
PyMC variables sampled (value space)
  sigma_log__               log(sigma)
  Intercept                 a_c
  x                         b_x
  1|school_sigma_log__      log(sd_school)
  1|school_offset           z
```

```
     sigma ~ HalfNormal(0, 1)          1|g_offset ~ Normal(0, 1)
 Intercept ~ Normal(0, 5)                     1|g ~ Deterministic(f(1|g_offset,
     1|g_sigma))
         x ~ Normal(0, 2)                    mu ~ Deterministic(f(Intercept,
             1|g_offset, x, 1|g_sigma))
 1|g_sigma ~ HalfNormal(0, 1)                 y ~ Normal(mu, sigma)
```

Two facts a researcher must know and would not see in Bambi's own printout. First, the `Intercept` in the PyMC model is a_c, the intercept at the predictor means; the `Intercept` in the returned trace is $a = a_c - \bar{x}\, b$. The prior Normal$(0, 5)$ is on a_c. With the example data $\bar{x} = -0.092$ and $b \approx 0.49$ the two differ by 0.045, small here and not small in general. Second, the sampler moves $\log \sigma$ and $\log s_g$, not σ and s_g; the joint density in that space carries the Jacobian $+\log \sigma + \log s_g$, which is what the reference below reproduces.

4 Log density and meaning (checks A1–A7)

```
65      y, Xc, _, gidx, levels = build_design
            (data, response, predictors,
            group, center)
66      a, b = float(point["a_c"]), np.
            atleast_1d(point["b"])
67      log_sigma, log_sd = float(point["
            log_sigma"]), float(point["log_sd
            "])
68      z = np.asarray(point["z"], dtype=
            float)
69      assert z.shape == (len(levels),)
70      sigma, sd = np.exp(log_sigma), np.exp
            (log_sd)
71
72      mu = a + Xc @ b + sd * z[gidx]
73      lp = np.sum(stats.norm.logpdf(y, loc=
            mu, scale=sigma))
74      lp += np.sum(stats.norm.logpdf(z,
            0.0, 1.0))
75      lp += prior_logpdf(*priors["intercept
            "], a)
76      for i, p in enumerate(predictors):
77          lp += prior_logpdf(*priors["b"][p
                ], b[i])
78      lp += prior_logpdf(*priors["sd"], sd)
79      lp += prior_logpdf(*priors["sigma"],
            sigma)
80      if jacobian:
81          lp += log_sigma + log_sd
```

`tests/reference_logp.py:65-81`. The reference builds its own centred design and its own sorted level index from the data frame, evaluates every prior with SciPy (half-Student-t written out), and adds the log-transform Jacobian. It shares no code with the library, so agreement is evidence about Bambi's graph, not about the layer.

The comparison point is a dictionary in PyMC's value space produced by `Model.value_point()`, whose keys are checked against `pymc_model.value_vars`.

Check	What is compared	Tolerance	Result
A1	`compile_logp()` vs reference, 5 random points, 4 formula/centring variants	10^{-8} abs	pass
A2	same with `jacobian=False`; difference equals $\log\sigma + \log s_g$	10^{-8}, 10^{-12}	pass
A3	`compile_dlogp()` vs central differences of the reference, 3 points	rel 10^{-5}	pass
A4	JAX-compiled density (`get_jaxified_logp`) vs PyTensor path	10^{-8} abs	pass
A5	scales fixed by `pm.do`: negative Hessian equals closed-form precision at two points; maximiser equals exact posterior mean	10^{-8}, 10^{-4}	pass
A6	$X_{\text{model}} = x - \bar{x}$, Z one-hot in sorted level order, `mu` and `1|g` from the graph vs by hand	10^{-12}	pass
A7	17 unsupported formulas and 5 data/prior faults raise the named error	—	pass

Three PyMC 5.28.5 facts surfaced while making A4–A6 pass, all verified in source and none a model problem. `get_jaxified_logp(model, negative_logp=...)` has its flag named backwards: the default `True` returns the log density and `False` negates it. `compile_d2logp()` returns the *negative* Hessian (`pymc.pytensorf.hessian(negate_output=True)`), i.e. the precision matrix, which is why A5 compares it to $+P$. A deterministic such as `mu` is a graph of random variables; to evaluate it at a point in value space the RVs must first be substituted (`model.replace_rvs_by_values`). Each is recorded in the test that hit it.

5 Sampling (checks B1–B2)

B1: exact conditional posterior. With $\sigma = 0.3$ and $s_g = 0.5$ fixed through `pm.do` on the inspectable model, the posterior of $(a_c, b, z_1..z_6)$ is Gaussian with precision $P = \mathrm{diag}(1/25, 1/4, 1, \ldots, 1) + W^\top W/\sigma^2$, $W = [\mathbf{1},\ x - \bar{x},\ s_g Z]$. NUTS on that model (n = 60, J = 6) against the exact mean, with MCSE = exact SD/$\sqrt{\text{ESS}}$:

Committed test: seed 11, 4 chains × 2000 draws, threshold $\lvert z\rvert < 4$ and SD within 10%	exact mean	MCMC mean	exact SD	MCMC SD	ESS	z
a_c	1.2147	1.2289	0.2078	0.2149	1273	+2.45
b_x	0.4561	0.4565	0.0423	0.0423	3159	+0.48
$z_0 \ldots z_5$			0.437–0.456	0.450–0.467	1413–1503	−2.20 to −2.80

Seed check, 4 chains, 2000 tune; z for a_c, for the identified direction $a_c + s_g\bar{z}$, and for the weak direction $a_c - s_g\bar{z}$							
seed	draws	ESS a_c	ESS ident.	ESS weak	$z(a_c)$	z(ident.)	z(weak)
11	2000	1315	8929	1294	+0.06	−0.96	+0.09
12	4000	2493	17328	2444	−1.69	+1.72	−1.76
13	4000	2474	17622	2429	+1.59	+0.89	+1.57
14	8000	4884	31828	4793	−1.14	−0.15	−1.13

The committed run passed its threshold ($|z| < 4$, SDs within 10%) but its eight z-scores share one sign pattern, so they are one event, not eight. The pattern is the direction $a_c - s_g\bar{z}$: the data pin down $a_c + s_g\bar{z}$ (the grand mean, exact SD 0.040, about one effective draw per draw) and leave $a_c - s_g\bar{z}$ to the priors (exact SD 0.41, about one effective draw per six). Across seeds and lengths the deviations alternate sign and stay under 1.8 MCSE, so the first run was a 2.5σ draw and not a bias. Files: `B1_conditional_gaussian.json`, `B1_seed_check.json`.

B2: simulated recovery on both backends. $n = 200$, $J = 20$, truth $a = 1.0$, $b = 0.5$, $s_g = 0.5$, $\sigma = 0.3$; 4 chains, 1000 tune, 2500 draws, `target_accept` 0.9, seed 7.

	mean	SD	94% HDI	$\hat{R}$	bulk ESS	tail ESS
PyMC NUTS, CPU: 0 divergences of 10000, E-BFMI 0.70, tree-depth saturation 0%						
Intercept	0.776	0.143	[0.491, 1.037]	1.003	755	1384
b_x	0.486	0.024	[0.441, 0.529]	1.000	5055	5721
s_g	0.642	0.113	[0.450, 0.858]	1.003	1215	2544
σ	0.306	0.016	[0.276, 0.336]	1.001	5208	6235
NumPyro NUTS via JAX, CPU: 0 divergences of 10000, E-BFMI 0.67, tree-depth saturation 0%						
Intercept	0.779	0.146	[0.518, 1.082]	1.005	771	1235
b_x	0.487	0.024	[0.442, 0.531]	1.001	5401	6061
s_g	0.647	0.117	[0.443, 0.862]	1.008	1010	1966
σ	0.305	0.017	[0.274, 0.337]	1.000	5229	6127

Every panel row passes on both backends and the truth is inside every 94% HDI. The two backends' posterior means differ by 0.0027 (Intercept), 0.0009 (b), 0.0052 (s_g), 0.0004 (σ), against 4-combined-MCSE tolerances of 0.030, 0.0019, 0.019, 0.0013: the same posterior, sampled two ways.

Two readings that would be wrong. The Intercept posterior sits at 0.78 against a nominal 1.0 and s_g at 0.64 against 0.5: this is not shrinkage error. The twenty simulated effects have realised mean −0.234 and realised SD 0.602, so the quantities the data can identify are $1.0 - 0.234 = 0.766$ and about 0.60, both near the posterior centres. And at 4 chains × 1000 draws these same fits gave $\hat{R} = 1.02$ and bulk ESS 350 on the Intercept on both backends, for the reason B1 exhibits: the weakly identified intercept-versus-mean-offset direction. The runs were lengthened; the thresholds were not loosened. One recovery on one simulated data set is not a calibration claim.

6 Devices: executed versus supported

Path	Status	Evidence
`backend="pymc"`, CPU	executed	B2, `example_pymc_cpu.txt` (wall 5.0 s incl. compile)
`backend="numpyro"`, `device="cpu"`	executed, 4 parallel CPU chains, float64	B2, `example_numpyro_cpu.txt` (3.9 s)
`backend="numpyro"`, `device="gpu"`	supported in code, **not validated**	`example_numpyro_gpu_attempt.txt`: the refusal
`backend="pymc"`, `device="gpu"`	refused by design	`test_gpu_on_pymc_backend_is_error`

```
70      if device == "cpu":
71          # Must run before the JAX backend
                initialises, else it is a no
                -op.
72          numpyro.set_host_device_count(int
                (chains))
73      jax.config.update("jax_enable_x64",
            True)
74      devs = jax.devices()
75      if device == "gpu":
76          gpus = [d for d in devs if d.
                platform in _GPU_PLATFORMS]
77          if not gpus:
78              raise DeviceError(
79                  "device='gpu' requested
                        for backend='numpyro
                        ', but JAX sees no
                        GPU. "
80                  f"Visible devices: {
                        _jax_devices_summary(
                        jax)}. Install a CUDA
                        -enabled "
81                  "jaxlib on a machine with
                         an NVIDIA GPU, or
                        pass device='cpu'. "
82                  "AutoBayes does not fall
                        back to CPU silently.
                        "
83              )
84          return gpus[0], "vectorized"
```

`autobayes/inference.py:70-84`. The report on every fit records the device the arrays actually landed on (probed inside `jax.default_device`), the dtype, and the chain method; on this machine it reads `cpu:cpu(id=0)`, `float64`, `parallel`. The two wall-clock figures above are not a speed comparison: same machine, same targets, but compilation is inside both numbers and the sizes are tiny.

The GPU command is in `docs/m0-gpu-validation.md` and `cluster/m0_gpu_smoke.sbatch`; the cluster's existing `electit_env2` lacks Bambi, so it needs a new environment. Nothing was submitted, per the handoff. What a GPU run would add is one fact: that the same compiled graph runs on the other device. It would not add a speed claim.

7 Prediction meaning (check A8)

In-sample `mu` from `predict()` equals $a + xb + u[g]$ computed by hand from the trace to 10^{-10}; `y` draws scatter around it with pooled SD equal to the posterior mean of σ. For an unseen group, `new_groups="population"` draws $u_{\text{new}} \sim \text{Normal}(0, s_g)$ per posterior draw, so its predictive SD is about $\sqrt{\sigma^2 + s_g^2}$: in the example 0.74 for a new school against 0.32 for an observed one

($\sqrt{0.306^2 + 0.642^2} = 0.71$ before posterior spread in a and s_g). `"resample_observed"` reproduces Bambi's behaviour, verified draw by draw to be one of the observed levels' effects; it is brms's default and it understates between-group variance, which is why it is not the default here.

8 Not claimed, and next

Not claimed: any family beyond Gaussian, varying slopes, multiple or nested grouping factors, categorical predictors, missing data, calibration in general, GPU execution, or any speedup. Follow-ups worth a decision: run `cluster/m0_gpu_smoke.sbatch` once a Bambi-capable environment exists on the cluster; decide whether the three PyMC quirks belong in `reference/libraries.md` (the pack is a fixed input, so that is Andrew's call); and choose the next family (M1) only after this review.